

UNIVERSIDAD SANTO TOMAS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA

Diseño e implementación de una base de datos para una clínica veterinaria en PostgreSQL

Adrian Grateron

Harold Rodriguez

Jhonatan Smith Doncel

Facultad de estadística

Universidad Santo Tomas

Seccional Bogotá

Bases de Datos I

Carlos Isaac Zainea Maya

Julio 2026

Contenido

Introducción.....3

1. Diseño conceptual.....3

    1.1 Análisis del caso y reglas de negocio.....3

    1.2 Lectura y simbología del modelo.....7

2. Diseño lógico.....8

3. Normalización.....10

    3.1 Dependencias funcionales.....10

    3.2 Verificación de 1NF, 2NF y 3NF.....10

    3.3 Anomalías evitadas.....11

4. Implementación en PostgreSQL.....11

    4.1 Criterios de implementación.....11

    4.2 Código DDL y reglas de integridad.....12

Conclusiones.....19

Referencias.....20

Introducción

Una clínica veterinaria no solo administra nombres y fechas: conserva una historia clínica que debe poder reconstruirse desde el primer contacto con el dueño hasta el tratamiento y la vacunación de la mascota. Cuando esta información se guarda sin una estructura definida, pueden aparecer duplicidades, datos contradictorios y dificultades para establecer quién atendió al paciente, qué diagnóstico se emitió o qué medicamento fue prescrito.

El modelo propuesto organiza ese recorrido de manera progresiva. El dueño registra una o más mascotas y puede suministrar varios teléfonos. Cada mascota puede asistir a múltiples citas, mientras que cada cita se asigna a un veterinario. Cuando la cita es atendida, puede producir un diagnóstico; el diagnóstico puede generar varias prescripciones y cada prescripción indica un medicamento concreto. De forma paralela, la mascota conserva un historial de aplicaciones de vacunas.

Recorrido principal: DUEÑO → MASCOTA → CITA → DIAGNÓSTICO → PRESCRIPCIÓN → MEDICAMENTO. Recorrido preventivo: MASCOTA → REGISTRO DE VACUNACIÓN → VACUNA.

Objetivo general. Diseñar e implementar una base de datos relacional para la gestión clínica básica de una veterinaria, utilizando un modelo conceptual en notación de Chen, un esquema lógico normalizado en Tercera Forma Normal y código DDL compatible con PostgreSQL administrado mediante pgAdmin 4.

Alcance. La solución cubre dueños, teléfonos, mascotas, veterinarios, citas, diagnósticos, medicamentos, prescripciones, vacunas y registros de vacunación.

1. Diseño conceptual

El diseño conceptual describe el negocio antes de hablar de tablas o instrucciones SQL. Su finalidad es identificar qué objetos existen, qué atributos los caracterizan y qué relaciones permiten reconstruir la atención clínica. La notación de Chen diferencia visualmente entidades, relaciones, atributos, cardinalidades y dependencias de existencia.

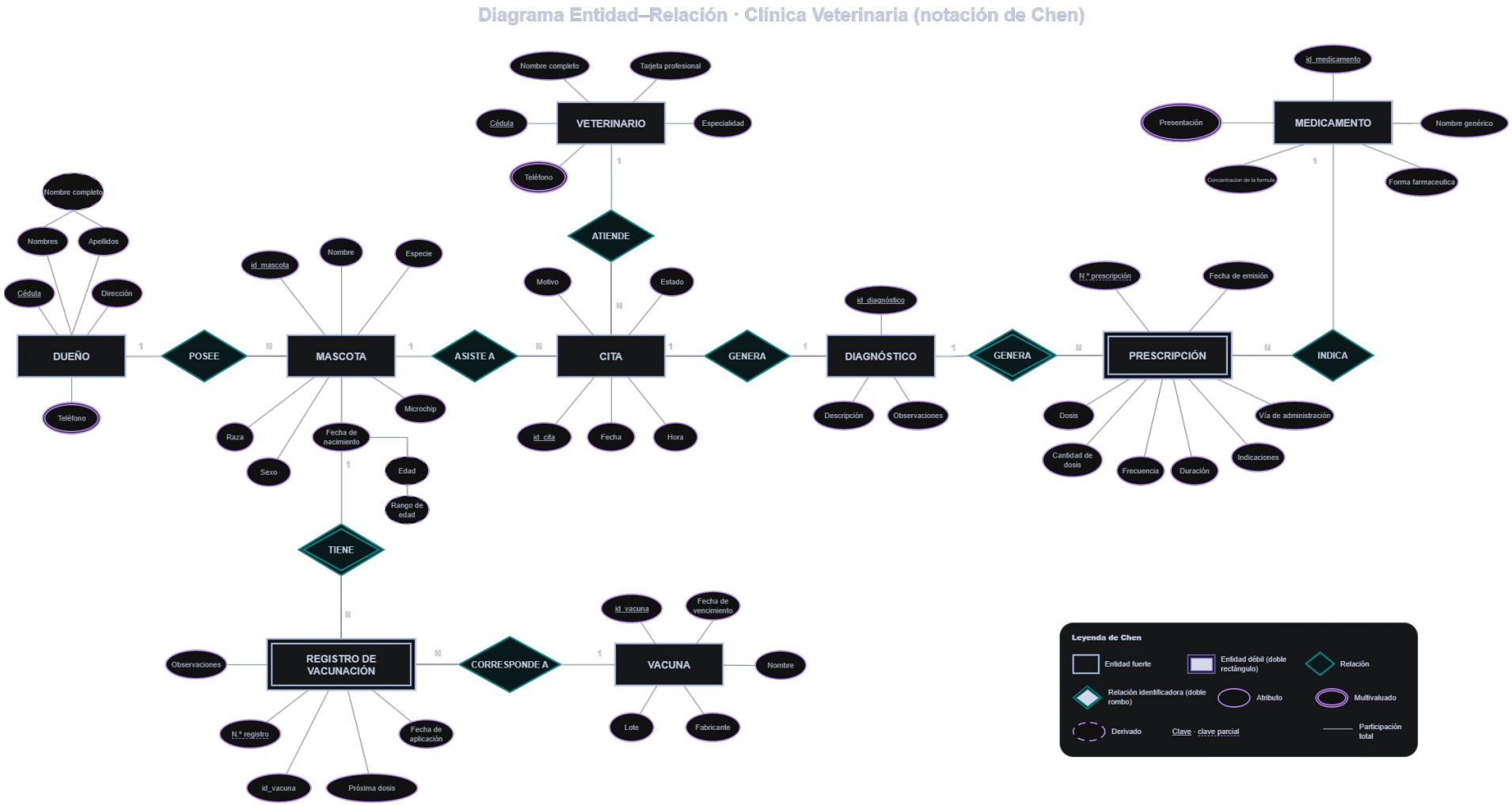
1.1 Análisis del caso y reglas de negocio

Regla de negocio	Cardinalidad	Interpretación
DUEÑO posee MASCOTA	1:N	Un dueño puede registrar varias mascotas; cada mascota pertenece a un único dueño.
DUEÑO tiene TELÉFONO	Multivaluado	Un dueño puede registrar más de un número de contacto.
MASCOTA asiste a CITA	1:N	Una mascota puede tener muchas citas; cada cita corresponde a una sola mascota.
VETERINARIO atiende CITA	1:N	Un veterinario puede atender varias citas; cada cita se asigna a un único profesional.
CITA genera DIAGNÓSTICO	0..1:1	Una cita atendida puede producir un diagnóstico y cada diagnóstico pertenece a una sola cita.
DIAGNÓSTICO genera PRESCRIPCIÓN	1:N	Un diagnóstico puede no requerir medicamentos o puede originar varias prescripciones.
PRESCRIPCIÓN indica MEDICAMENTO	N:1	Cada prescripción representa un medicamento concreto; un medicamento puede aparecer en muchas prescripciones.
MASCOTA tiene REGISTRO DE VACUNACIÓN	1:N	La mascota puede acumular múltiples eventos de vacunación durante su vida.
REGISTRO corresponde a VACUNA	N:1	Cada aplicación referencia una vacuna específica; una vacuna puede aparecer en múltiples registros.

La relación entre CITA y DIAGNÓSTICO se interpreta como “máximo uno”. Una cita cancelada, programada o marcada como no asistida no genera diagnóstico; una cita atendida debe producirlo. En el modelo relacional se impide que una cita tenga dos diagnósticos mediante una restricción de unicidad sobre id_cita.

PRESCRIPCIÓN y REGISTRO DE VACUNACIÓN se modelan como entidades débiles. El número de prescripción solo tiene sentido dentro de un diagnóstico, y el número de registro solo identifica un evento dentro de la historia de una mascota. Por ello, ambas entidades se identifican mediante claves compuestas.

Figura 1. Diagrama Entidad-Relación en notación de Chen



El diagrama representa el flujo clínico y preventivo, e identifica las entidades débiles mediante doble rectángulo y sus relaciones identificadoras mediante doble rombo.

1.2 Lectura y simbología del modelo

Símbolo	Significado	Aplicación
Rectángulo simple	Entidad fuerte	DUEÑO, MASCOTA, VETERINARIO, CITA, DIAGNÓSTICO, MEDICAMENTO y VACUNA.
Rectángulo doble	Entidad débil	PRESCRIPCIÓN y REGISTRO DE VACUNACIÓN.
Rombo simple	Relación regular	POSEE, ASISTE A, ATIENDE, GENERA entre cita y diagnóstico, INDICA y CORRESPONDE A.
Rombo doble	Relación identificadora	GENERA entre DIAGNÓSTICO y PRESCRIPCIÓN, y TIENE entre MASCOTA y REGISTRO DE VACUNACIÓN.
Óvalo simple	Atributo	Datos descriptivos como nombre, fecha, motivo o especialidad.
Óvalo doble	Atributo multivaluado	Teléfono del dueño.
Óvalo discontinuo	Atributo derivado	Edad de la mascota, calculada desde fecha_nacimiento.
Subrayado continuo	Clave	Identificador de una entidad fuerte.
Subrayado discontinuo	Clave parcial	Número de prescripción y número de registro.

La edad aparece en el modelo conceptual como atributo derivado, pero no se almacena en la base. Guardar la edad produciría un dato que se desactualiza con el tiempo; por eso se conserva únicamente la fecha de nacimiento y la edad se calcula cuando se consulta.

En PRESCRIPCIÓN, una fila representa una orden de un medicamento dentro de un diagnóstico. La propia entidad contiene dosis, unidad, frecuencia, vía de administración, duración e indicaciones, por lo que no se requiere una segunda tabla de detalle para el alcance de este trabajo.

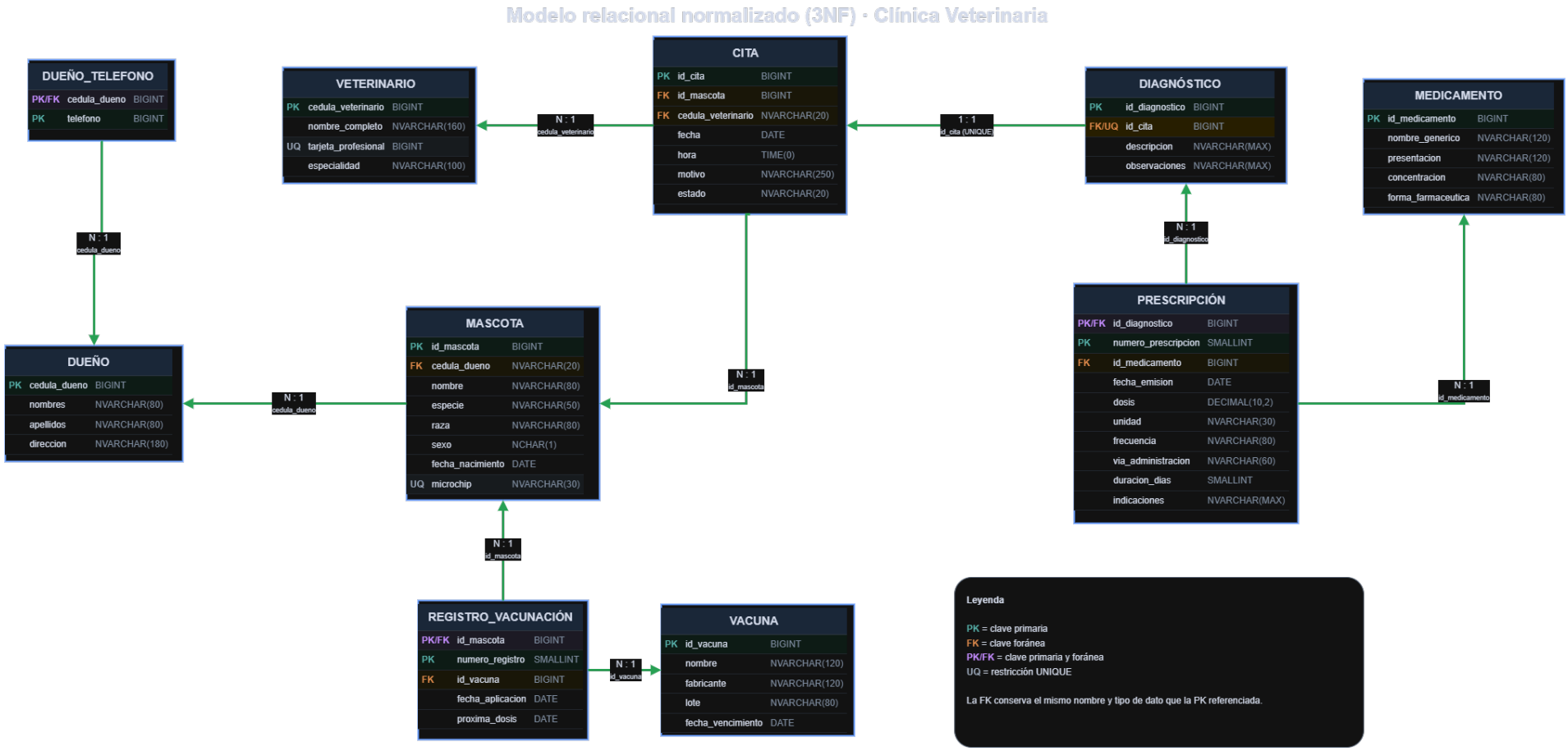
En vacunación, REGISTRO DE VACUNACIÓN no repite el nombre, fabricante ni lote. Esa información se consulta desde VACUNA mediante id_vacuna, evitando duplicar datos en cada aplicación.

2. Diseño lógico

El diseño lógico transforma el modelo conceptual en tablas relacionadas. Las entidades fuertes se convierten en tablas con clave primaria propia; las entidades débiles usan claves compuestas; y las relaciones se materializan mediante claves foráneas. Cuando una PK migra a otra tabla como FK conserva el mismo nombre y tipo de dato, de modo que la conexión sea explícita.

Elemento conceptual	Transformación relacional
Entidad fuerte	Tabla con clave primaria propia.
Entidad débil	Tabla cuya PK combina la clave de la entidad propietaria y la clave parcial.
Atributo compuesto	Nombre completo del dueño se separa en nombres y apellidos.
Atributo multivaluado	Teléfono se convierte en DUENO_TELEFONO.
Relación 1:N	La PK del lado 1 se copia como FK en la tabla del lado N.
Relación 1:1	DIAGNOSTICO almacena id_cita como FK y UNIQUE.
Atributo derivado	Edad no se almacena; se calcula desde fecha_nacimiento.

Figura 2. Modelo relacional con claves, tipos y cardinalidades



Las PK, FK, claves compuestas y restricciones de unicidad materializan las relaciones definidas en el modelo conceptual.

3. Normalización

Normalizar significa organizar la información de modo que cada hecho se almacene una sola vez y dependa de la clave correcta. El objetivo es reducir redundancias y evitar anomalías de inserción, actualización y eliminación.

3.1 Dependencias funcionales

Determinante	Atributos determinados
cedula_dueno	nombres, apellidos, direccion
id_mascota	cedula_dueno, nombre, especie, raza, sexo, fecha_nacimiento, microchip
cedula_veterinario	nombre_completo, tarjeta_profesional, especialidad
id_cita	id_mascota, cedula_veterinario, fecha, hora, motivo, estado
id_diagnostico	id_cita, descripcion, observaciones
id_medicamento	nombre_generico, presentacion, concentracion, forma_farmaceutica
(id_diagnostico, numero_prescripcion)	id_medicamento, fecha_emision, dosis, unidad, frecuencia, via_administracion, duracion_dias, indicaciones
id_vacuna	nombre, fabricante, lote, fecha_vencimiento
(id_mascota, numero_registro)	id_vacuna, fecha_aplicacion, proxima_dosis

3.2 Verificación de 1NF, 2NF y 3NF

Forma normal	Justificación
Primera Forma Normal (1NF)	Cada columna contiene valores atómicos. Los teléfonos no se almacenan juntos en DUENO, sino como filas independientes en DUENO_TELEFONO. La edad no se persiste; se deriva de fecha_nacimiento.
Segunda Forma Normal (2NF)	Las tablas con clave compuesta son DUENO_TELEFONO, PRESCRIPCION y REGISTRO_VACUNACION. Sus atributos no clave dependen de la clave completa y no solo de una parte.
Tercera Forma Normal (3NF)	No existen dependencias transitivas entre atributos no clave. Los datos del dueño no se repiten en MASCOTA, los del veterinario no se repiten en CITA, los del medicamento no se copian en PRESCRIPCION y los de la vacuna no se duplican en REGISTRO_VACUNACION.

Conclusión de normalización. El esquema cumple como mínimo la Tercera Forma Normal. La separación de teléfonos, medicamentos y vacunas reduce duplicidades y facilita el mantenimiento de la información.

3.3 Anomalías evitadas

Riesgo	Control aplicado
Actualización	Cambiar la presentación o concentración de un medicamento se realiza una sola vez en MEDICAMENTO.
Inserción	Es posible registrar un medicamento o una vacuna antes de que sean utilizados, sin inventar una cita o una mascota.
Eliminación	Eliminar una prescripción no borra el medicamento del catálogo; eliminar un dueño con mascotas registradas está restringido.
Inconsistencia	UNIQUE sobre DIAGNOSTICO.id_cita impide dos diagnósticos para la misma cita.
Datos derivados	La edad no se desactualiza porque no se almacena; se calcula desde fecha_nacimiento.

4. Implementación en PostgreSQL

La implementación física traduce el esquema lógico a instrucciones SQL ejecutables en PostgreSQL mediante pgAdmin 4. Se aplican restricciones para proteger la identidad, obligatoriedad, unicidad, dominio e integridad referencial. Además, se crean índices sobre claves foráneas y dos triggers (condiciones sobre la operación) para validar reglas que dependen de más de una tabla.

4.1 Criterios de implementación

Mecanismo	Función dentro del modelo
PRIMARY KEY	Identifica cada fila; incluye claves compuestas en las entidades débiles.
FOREIGN KEY	Impide registrar referencias a entidades inexistentes.
NOT NULL	Obliga a capturar datos indispensables.
UNIQUE	Evita duplicar tarjeta profesional, microchip, diagnóstico por cita y combinaciones de catálogos.
CHECK	Restringe sexo, estados, dosis, consecutivos y fechas.
Índices	Aceleran las uniones sobre claves foráneas.
Triggers	Impiden aplicar una vacuna vencida y emitir una prescripción

Mecanismo	Función dentro del modelo
	antes de la cita.

La relación CITA-DIAGNÓSTICO se implementa mediante una FK obligatoria y una restricción UNIQUE sobre diagnostico.id_cita. Esto garantiza que un diagnóstico pertenezca a una cita y que una cita tenga como máximo un diagnóstico.

4.2 Código DDL y reglas de integridad

```
-- CLÍNICA VETERINARIA
--   * Las PK y FK relacionadas conservan exactamente el mismo tipo de dato


BEGIN;

DROP SCHEMA IF EXISTS clinica_veterinaria CASCADE;
CREATE SCHEMA clinica_veterinaria;
SET search_path TO clinica_veterinaria, public;


-- 1. TABLAS PRINCIPALES


CREATE TABLE dueno (
    cedula_dueno  BIGINT       NOT NULL,
    nombres       VARCHAR(80)  NOT NULL,
    apellidos     VARCHAR(80)  NOT NULL,
    direccion     VARCHAR(180) NOT NULL,

    CONSTRAINT pk_dueno
        PRIMARY KEY (cedula_dueno),

    CONSTRAINT ck_dueno_cedula_positiva
        CHECK (cedula_dueno > 0),

    CONSTRAINT ck_dueno_nombres_no_vacios
        CHECK (BTRIM(nombres) <> ''),

    CONSTRAINT ck_dueno_apellidos_no_vacios
        CHECK (BTRIM(apellidos) <> ''),

    CONSTRAINT ck_dueno_direccion_no_vacia
        CHECK (BTRIM(direccion) <> '')
);


CREATE TABLE dueno_telefono (
    cedula_dueno  BIGINT      NOT NULL,
    telefono      VARCHAR(25) NOT NULL,

    CONSTRAINT pk_dueno_telefono
        PRIMARY KEY (cedula_dueno, telefono),

    CONSTRAINT fk_dueno_telefono_dueno
        FOREIGN KEY (cedula_dueno)
```

```
        REFERENCES dueno (cedula_dueno)
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT ck_dueno_telefono_formato
        CHECK (telefono ~ '^[0-9+() -]{7,25}$')
);


CREATE TABLE veterinario (
    cedula_veterinario   BIGINT       NOT NULL,
    nombre_completo      VARCHAR(160) NOT NULL,
    tarjeta_profesional  BIGINT       NOT NULL,
    especialidad         VARCHAR(100) NOT NULL,

    CONSTRAINT pk_veterinario
        PRIMARY KEY (cedula_veterinario),

    CONSTRAINT uq_veterinario_tarjeta_profesional
        UNIQUE (tarjeta_profesional),

    CONSTRAINT ck_veterinario_cedula_positiva
        CHECK (cedula_veterinario > 0),

    CONSTRAINT ck_veterinario_tarjeta_positiva
        CHECK (tarjeta_profesional > 0),

    CONSTRAINT ck_veterinario_nombre_no_vacio
        CHECK (BTRIM(nombre_completo) <> ''),

    CONSTRAINT ck_veterinario_especialidad_no_vacia
        CHECK (BTRIM(especialidad) <> '')
);


CREATE TABLE mascota (
    id_mascota        BIGINT GENERATED BY DEFAULT AS IDENTITY,
    cedula_dueno      BIGINT      NOT NULL,
    nombre            VARCHAR(80) NOT NULL,
    especie           VARCHAR(50) NOT NULL,
    raza              VARCHAR(80),
    sexo              CHAR(1)     NOT NULL,
    fecha_nacimiento  DATE        NOT NULL,
    microchip         BIGINT,

    CONSTRAINT pk_mascota
        PRIMARY KEY (id_mascota),

    CONSTRAINT fk_mascota_dueno
        FOREIGN KEY (cedula_dueno)
        REFERENCES dueno (cedula_dueno)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT uq_mascota_microchip
        UNIQUE (microchip),

    CONSTRAINT ck_mascota_nombre_no_vacio
        CHECK (BTRIM(nombre) <> ''),

    CONSTRAINT ck_mascota_especie_no_vacia
```

```
        CHECK (BTRIM(especie) <> ''),

    CONSTRAINT ck_mascota_raza_no_vacia
        CHECK (raza IS NULL OR BTRIM(raza) <> ''),

    CONSTRAINT ck_mascota_sexo
        CHECK (sexo IN ('M', 'F', 'O')),

    CONSTRAINT ck_mascota_fecha_nacimiento
        CHECK (fecha_nacimiento <= CURRENT_DATE),

    CONSTRAINT ck_mascota_microchip_positivo
        CHECK (microchip IS NULL OR microchip > 0)
);


CREATE TABLE cita (
    id_cita               BIGINT GENERATED BY DEFAULT AS IDENTITY,
    id_mascota            BIGINT       NOT NULL,
    cedula_veterinario    BIGINT       NOT NULL,
    fecha                 DATE         NOT NULL,
    hora                  TIME(0)      NOT NULL,
    motivo                VARCHAR(250) NOT NULL,
    estado                VARCHAR(20)  NOT NULL DEFAULT 'PROGRAMADA',

    CONSTRAINT pk_cita
        PRIMARY KEY (id_cita),

    CONSTRAINT fk_cita_mascota
        FOREIGN KEY (id_mascota)
        REFERENCES mascota (id_mascota)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_cita_veterinario
        FOREIGN KEY (cedula_veterinario)
        REFERENCES veterinario (cedula_veterinario)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT uq_cita_veterinario_fecha_hora
        UNIQUE (cedula_veterinario, fecha, hora),

    CONSTRAINT uq_cita_mascota_fecha_hora
        UNIQUE (id_mascota, fecha, hora),

    CONSTRAINT ck_cita_motivo_no_vacio
        CHECK (BTRIM(motivo) <> ''),

    CONSTRAINT ck_cita_estado
        CHECK (
            estado IN (
                'PROGRAMADA',
                'CONFIRMADA',
                'ATENDIDA',
                'CANCELADA',
                'NO_ASISTIO'
            )
        )
);
```

```
CREATE TABLE diagnostico (
    id_diagnostico  BIGINT GENERATED BY DEFAULT AS IDENTITY,
    id_cita         BIGINT NOT NULL,
    descripcion     TEXT   NOT NULL,
    observaciones   TEXT,

    CONSTRAINT pk_diagnostico
        PRIMARY KEY (id_diagnostico),

    CONSTRAINT fk_diagnostico_cita
        FOREIGN KEY (id_cita)
        REFERENCES cita (id_cita)
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    -- Garantiza que una cita tenga como máximo un diagnóstico.
    CONSTRAINT uq_diagnostico_id_cita
        UNIQUE (id_cita),

    CONSTRAINT ck_diagnostico_descripcion_no_vacia
        CHECK (BTRIM(descripcion) <> '')
);


CREATE TABLE medicamento (
    id_medicamento      SMALLINT GENERATED BY DEFAULT AS IDENTITY,
    nombre_generico     VARCHAR(120) NOT NULL,
    presentacion        VARCHAR(120) NOT NULL,
    concentracion       VARCHAR(80)  NOT NULL,
    forma_farmaceutica  VARCHAR(80)  NOT NULL,

    CONSTRAINT pk_medicamento
        PRIMARY KEY (id_medicamento),

    CONSTRAINT uq_medicamento_descripcion
        UNIQUE (
            nombre_generico,
            presentacion,
            concentracion,
            forma_farmaceutica
        ),

    CONSTRAINT ck_medicamento_id_positivo
        CHECK (id_medicamento > 0),

    CONSTRAINT ck_medicamento_nombre_no_vacio
        CHECK (BTRIM(nombre_generico) <> ''),

    CONSTRAINT ck_medicamento_presentacion_no_vacia
        CHECK (BTRIM(presentacion) <> ''),

    CONSTRAINT ck_medicamento_concentracion_no_vacia
        CHECK (BTRIM(concentracion) <> ''),

    CONSTRAINT ck_medicamento_forma_no_vacia
        CHECK (BTRIM(forma_farmaceutica) <> '')
);


-- Entidad débil:
```

```
-- PK completa = (id_diagnostico, numero_prescripcion).
CREATE TABLE prescripcion (
    id_diagnostico       BIGINT   NOT NULL,
    numero_prescripcion  SMALLINT NOT NULL,
    id_medicamento       SMALLINT NOT NULL,
    fecha_emision        DATE     NOT NULL DEFAULT CURRENT_DATE,
    dosis                NUMERIC(10,2) NOT NULL,
    unidad               VARCHAR(30)   NOT NULL,
    frecuencia           VARCHAR(80)   NOT NULL,
    via_administracion   VARCHAR(60)   NOT NULL,
    duracion_dias        SMALLINT,
    indicaciones         TEXT,

    CONSTRAINT pk_prescripcion
        PRIMARY KEY (id_diagnostico, numero_prescripcion),

    CONSTRAINT fk_prescripcion_diagnostico
        FOREIGN KEY (id_diagnostico)
        REFERENCES diagnostico (id_diagnostico)
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_prescripcion_medicamento
        FOREIGN KEY (id_medicamento)
        REFERENCES medicamento (id_medicamento)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_prescripcion_numero
        CHECK (numero_prescripcion > 0),

    CONSTRAINT ck_prescripcion_dosis
        CHECK (dosis > 0),

    CONSTRAINT ck_prescripcion_unidad_no_vacia
        CHECK (BTRIM(unidad) <> ''),

    CONSTRAINT ck_prescripcion_frecuencia_no_vacia
        CHECK (BTRIM(frecuencia) <> ''),

    CONSTRAINT ck_prescripcion_via_no_vacia
        CHECK (BTRIM(via_administracion) <> ''),

    CONSTRAINT ck_prescripcion_duracion
        CHECK (duracion_dias IS NULL OR duracion_dias > 0)
);


CREATE TABLE vacuna (
    id_vacuna         SMALLINT GENERATED BY DEFAULT AS IDENTITY,
    nombre            VARCHAR(120) NOT NULL,
    fabricante        VARCHAR(120) NOT NULL,
    lote              VARCHAR(80)  NOT NULL,
    fecha_vencimiento DATE         NOT NULL,

    CONSTRAINT pk_vacuna
        PRIMARY KEY (id_vacuna),

    CONSTRAINT uq_vacuna_lote
        UNIQUE (nombre, fabricante, lote),
```

```
    CONSTRAINT ck_vacuna_id_positivo
        CHECK (id_vacuna > 0),

    CONSTRAINT ck_vacuna_nombre_no_vacio
        CHECK (BTRIM(nombre) <> ''),

    CONSTRAINT ck_vacuna_fabricante_no_vacio
        CHECK (BTRIM(fabricante) <> ''),

    CONSTRAINT ck_vacuna_lote_no_vacio
        CHECK (BTRIM(lote) <> '')
);


-- Entidad débil:
-- PK completa = (id_mascota, numero_registro).
CREATE TABLE registro_vacunacion (
    id_mascota       BIGINT   NOT NULL,
    numero_registro  SMALLINT NOT NULL,
    id_vacuna        SMALLINT NOT NULL,
    fecha_aplicacion DATE     NOT NULL,
    proxima_dosis    DATE,

    CONSTRAINT pk_registro_vacunacion
        PRIMARY KEY (id_mascota, numero_registro),

    CONSTRAINT fk_registro_vacunacion_mascota
        FOREIGN KEY (id_mascota)
        REFERENCES mascota (id_mascota)
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_registro_vacunacion_vacuna
        FOREIGN KEY (id_vacuna)
        REFERENCES vacuna (id_vacuna)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT uq_registro_vacunacion_evento
        UNIQUE (id_mascota, id_vacuna, fecha_aplicacion),

    CONSTRAINT ck_registro_vacunacion_numero
        CHECK (numero_registro > 0),

    CONSTRAINT ck_registro_vacunacion_fecha
        CHECK (fecha_aplicacion <= CURRENT_DATE),

    CONSTRAINT ck_registro_vacunacion_proxima_dosis
        CHECK (
            proxima_dosis IS NULL
            OR proxima_dosis >= fecha_aplicacion
        )
);



-- 2. ÍNDICES SOBRE CLAVES FORÁNEAS


CREATE INDEX ix_mascota_cedula_dueno
    ON mascota (cedula_dueno);
```

```
CREATE INDEX ix_cita_id_mascota
    ON cita (id_mascota);

CREATE INDEX ix_cita_cedula_veterinario
    ON cita (cedula_veterinario);

CREATE INDEX ix_prescripcion_id_medicamento
    ON prescripcion (id_medicamento);

CREATE INDEX ix_registro_vacunacion_id_vacuna
    ON registro_vacunacion (id_vacuna);



-- 3. REGLAS DE NEGOCIO ENTRE TABLAS


-- 3.1. Impide registrar una vacuna aplicada después de su vencimiento.
CREATE OR REPLACE FUNCTION fn_validar_vacuna_vigente()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_fecha_vencimiento DATE;
BEGIN
    SELECT fecha_vencimiento
      INTO v_fecha_vencimiento
      FROM vacuna
     WHERE id_vacuna = NEW.id_vacuna;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'No existe la vacuna con id_vacuna = %',
            NEW.id_vacuna;
    END IF;

    IF NEW.fecha_aplicacion > v_fecha_vencimiento THEN
        RAISE EXCEPTION
            'La vacuna % venció el % y no puede aplicarse el %',
            NEW.id_vacuna,
            v_fecha_vencimiento,
            NEW.fecha_aplicacion;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_validar_vacuna_vigente
BEFORE INSERT OR UPDATE OF id_vacuna, fecha_aplicacion
ON registro_vacunacion
FOR EACH ROW
EXECUTE FUNCTION fn_validar_vacuna_vigente();


-- 3.2. Impide que una prescripción tenga fecha anterior a la cita.
CREATE OR REPLACE FUNCTION fn_validar_fecha_prescripcion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    v_fecha_cita DATE;
BEGIN
    SELECT c.fecha
      INTO v_fecha_cita
      FROM diagnostico d
      JOIN cita c
        ON c.id_cita = d.id_cita
     WHERE d.id_diagnostico = NEW.id_diagnostico;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'No fue posible determinar la cita del diagnóstico %',
            NEW.id_diagnostico;
    END IF;

    IF NEW.fecha_emision < v_fecha_cita THEN
        RAISE EXCEPTION
            'La prescripción no puede emitirse antes de la cita (%).',
            v_fecha_cita;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_validar_fecha_prescripcion
BEFORE INSERT OR UPDATE OF id_diagnostico, fecha_emision
ON prescripcion
FOR EACH ROW
EXECUTE FUNCTION fn_validar_fecha_prescripcion();
```

Conclusiones

El resultado no es únicamente un conjunto de tablas, sino una representación coherente del recorrido clínico de una mascota. El modelo conceptual permitió validar el significado del negocio antes de implementar tecnología; el modelo relacional materializó ese significado mediante claves y restricciones; y la normalización aseguró que cada hecho se almacenara en el lugar apropiado.

- La trazabilidad permite recorrer la información desde el dueño y la mascota hasta la cita, el diagnóstico, el medicamento formulado y la vacuna aplicada.
- PRESCRIPCION y REGISTRO_VACUNACION representan correctamente hechos dependientes de un diagnóstico y de una mascota, respectivamente.
- La Tercera Forma Normal reduce redundancias y separa los datos maestros de los eventos clínicos.
- El DDL de PostgreSQL protege la integridad mediante PK, FK, NOT NULL, UNIQUE, CHECK, índices y triggers.

- El poblado y las consultas de validación demuestran que el modelo puede reconstruir la atención y el historial preventivo sin ambigüedades.

Referencias

Elmasri, R., & Navathe, S. B. (2023). Fundamentals of Database Systems (8th ed.). Pearson.

Universidad Santo Tomás. (2026). Guía de actividades: Gestión de bases de datos. Caso de estudio “Clínica Veterinaria”.

Zainea, C. (2025). Material interactivo, Sección 2: Gestión de bases de datos. Universidad Santo Tomás.

Zainea, C. (2025). Video de profundización: De la realidad al modelo: el arte del diseño conceptual. Universidad Santo Tomás.